

Arkajit Ganguly

Roll: CrS 2504 • Assignment – 1

Course: Cryptography and Security Implementations

(1) An algorithm to compute the gcd of two integers

We choose to work with the Euclidean algorithm in its division form, as it is computationally equivalent to the subtractive form but much faster.

$a \bmod b$ is just repeated subtraction. Computing $1071 \bmod 462$ means subtracting 462 twice. So the division form does in one step what the subtractive form does in q steps. If the remainder sequence has k steps with quotients $q_1, \dots, q_k$, then the division form takes exactly k steps and the subtractive form takes exactly $\sum_{i=1}^k q_i$ subtractions. Every $q_i \geq 1$, so $k \leq \sum q_i$, and they are equal only when all the quotients are 1.

Take $\gcd(2^{32} - 1, 1)$. The division form takes 1 step, the subtractive form takes 4,294,967,294 subtractions. The reason is that k is bounded by the bit length n , which we prove below, but $\sum q_i$ is bounded only by the value 2^n . So the subtractive form is exponential in the input size.

There is one more reason. On an unsigned type, computing $a - b$ when $a < b$ wraps modulo 2^n and gives a huge number. The division form never has this problem, because $a \bmod b$ always lies in $[0, b)$.

The lemma we need

For $b > 0$, $\gcd(a, b) = \gcd(b, a \bmod b)$.

Proof. Write $a = qb + r$ with $0 \leq r < b$.

If $d \mid a$ and $d \mid b$, then $d \mid (a - qb) = r$, so d is a common divisor of b and r .

If $e \mid b$ and $e \mid r$, then $e \mid (qb + r) = a$, so e is a common divisor of a and b .

So the pair (a, b) and the pair (b, r) have exactly the same set of common divisors, hence the same greatest one. ■

The algorithm terminates because $r = a \bmod b$ satisfies $0 \leq r < b$, so the second argument strictly decreases and must reach 0. And $\gcd(a, 0) = a$, since every integer divides 0, so the common divisors of $(a, 0)$ are just the divisors of a , the largest being a .

The algorithm (without recursion)

```
function GCD(a, b) {
  while b != 0 {
    r ← a mod b;
    a ← b;
    b ← r;
  } end while;
  return a;
} end function;
```

The C code

```
#include <stdio.h>

unsigned int gcd(unsigned int a, unsigned int b)
```

```

{
    while (b != 0) {
        unsigned int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main(void)
{
    printf("%u\n", gcd(1071, 462));
    return 0;
}

```

How the code works

$\text{gcd}(a, b)$ does not change in any iteration, since we replace (a, b) by $(b, a \bmod b)$ and that is exactly the lemma. Also b strictly decreases every iteration, so the loop ends. When it ends $b = 0$, and $\text{gcd}(a, 0) = a$, so we return a .

Trace of $\text{gcd}(1071, 462)$:

step	a	b	q	$r = a \bmod b$
1	1071	462	2	147
2	462	147	3	21
3	147	21	7	0
	21	0	—	return 21

If $a < b$ we do not need a swap. $a \bmod b = a$ when $a < b$, so the first iteration performs the swap by itself. It costs one extra division, and nothing breaks.

Complexity analysis

$$\begin{aligned}
 r_0 &= q_1 r_1 + r_2 \\
 &= q_1 (q_2 r_2 + r_3) + r_2 \\
 &= (q_1 q_2 + 1) r_2 + q_1 r_3
 \end{aligned}$$

and since $q_1, q_2 \geq 1$,

$$q_1 q_2 + 1 \geq 2, \quad \text{hence} \quad r_0 \geq 2r_2.$$

Thus at every two steps, at least one bit of the remainder is lost. Starting from $r_0 < 2^n$ and terminating when the remainder hits 0,

$$\boxed{\# \text{steps} \leq 2n}$$

Dividing an n -bit number by an m -bit number, producing an ℓ -bit quotient, costs $O(\ell \cdot m)$ bit operations. Step i divides r_{i-1} by r_i producing q_i . Thus

$$\text{cost} \approx (\log_2 q_i + 1) \cdot n \text{ bit operations.}$$

Thus total cost,

$$O\left(n \sum_{i=1}^K (\log_2 q_i + 1)\right) = O\left(n \left(\log_2 \prod_{i=1}^K q_i + K\right)\right).$$

We had,

$$r_0 \geq q_1 r_1 \geq q_1 q_2 r_2 \geq \cdots \geq \left(\prod_i q_i \right) r_K \geq \prod_i q_i.$$

So, $\log_2 \prod_i q_i \leq n$. Hence,

$$\boxed{O(n^2) \text{ bit operations.}}$$

(2) Essay: how to add two integers

An n -bit unsigned integer a is a string of bits $a_{n-1}a_{n-2}\dots a_1a_0$ standing for

$$a = \sum_{i=0}^{n-1} a_i 2^i, \quad a_i \in \{0, 1\}.$$

The number is not one object. It is a sequence of digits in base 2. So addition has to be defined the way we define it for digits.

The rule for one position

At position i three bits come in: a_i , b_i , and the carry c_i from the position below. Their sum is between 0 and 3, so it needs two bits going out, the sum bit s_i and the carry-out c_{i+1} :

$$a_i + b_i + c_i = s_i + 2c_{i+1}.$$

All 8 cases:

a_i	b_i	c_i	s_i	c_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

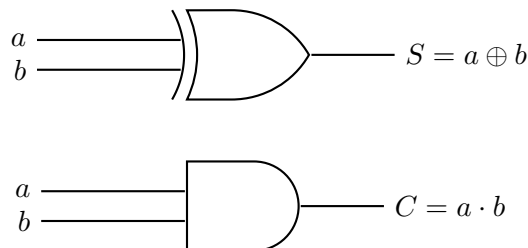
Reading the table off,

$$s_i = a_i \oplus b_i \oplus c_i, \quad c_{i+1} = a_i b_i + c_i(a_i \oplus b_i).$$

So s_i is 1 when an odd number of the three inputs are 1, which is exactly XOR, and c_{i+1} is 1 when at least two of them are 1, which is the majority function.

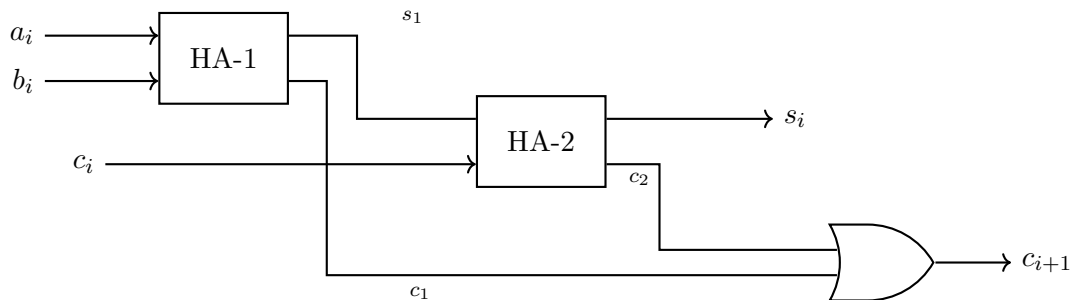
The half adder

This adds two bits, with no carry coming in.



The full adder

This adds three bits. It is two half adders plus an OR.



The final OR is correct because the two carries can never both be 1. If HA-1 produces a carry then $a_i = b_i = 1$, so its sum bit is 0, so HA-2 is adding $0 + c_i$ and cannot carry. So OR is safe here.

One full adder therefore costs $2 \text{ XOR} + 2 \text{ AND} + 1 \text{ OR} = 5$ gates. To add two n -bit numbers we put one full adder at each position and feed the carry-out of one into the carry-in of the next.

Watching the addition happen: $1011_2 + 0110_2$, that is $11 + 6$

position	a_i	b_i	c_i	$s_i = a_i \oplus b_i \oplus c_i$	c_{i+1}
0	1	0	0	1	0
1	1	1	0	0	1
2	0	1	1	0	1
3	1	0	1	0	1

The result is $c_4 s_3 s_2 s_1 s_0 = 10001_2 = 17$, which is correct.

The carry made at position 1 travelled all the way up to position 4, passing through every full adder in between. And $c_4 = 1$, but in a 4-bit machine there is no fifth wire, so that bit is dropped and we get $0001 = 1 = 17 \bmod 16$. That dropped carry is exactly the unsigned wraparound of C.

Why do we do it bit by bit?

First, a gate has two inputs. Hardware can only look at a constant number of bits at a time. There is no physical device that inspects a 32-bit number as one whole object.

Second, the alternative is exponential. Suppose we built a lookup table for n -bit addition instead. It would need one entry per input pair, that is

$$2^n \times 2^n = 2^{2n} \text{ entries.}$$

For $n = 32$ that is 2^{64} entries, which is impossible. The table is exponential in the input size even though the problem is trivial. Doing it bit by bit gives n positions $\times 5$ gates $= 5n$ gates, which is linear in n . This is the same lesson as in question (1): the cost must be measured in n , the number of bits, and not in the value 2^n .

Third, position i needs to know only one bit about everything below it, namely the carry. It does not need the whole lower half of the number. That single fact is what makes the problem split into positions at all.

Fourth, the carry flows only from low to high and never back. So one sweep starting at the LSB is enough.

Cost of addition

The gate count is $5n$, linear in n , and the bit complexity is $O(n)$. Addition is the linear-time primitive, and we measure multiplication and division against it. The chain has depth $O(n)$ because the carry has to cross all n stages; real adders cut this down using carry lookahead, but the bit complexity stays $O(n)$ either way.

(3) Complexity of division $\equiv$ complexity of multiplication

Let $M(n)$ be the cost of multiplying two n -bit numbers, and $D(n)$ the cost of dividing a $2n$ -bit number by an n -bit number. We claim $D(n) = \Theta(M(n))$. We assume the usual regularity conditions $M(n) \geq n$ and $M(2n) \geq 2M(n)$, that is, multiplication is at least linear and does not get cheaper as the numbers grow.

The easy observation first. Schoolbook multiplication is n rounds of shift and add, and schoolbook division is n rounds of shift, compare and subtract. Same shape, same count, both $\Theta(n^2)$. So at the schoolbook level they are visibly equal. But we want the statement for any multiplication algorithm, however clever, so we prove both directions.

First direction: $D(n) = O(M(n))$

Division reduces to computing a reciprocal, and the reciprocal is computed by Newton–Raphson using only multiplications. To find $1/b$ we apply Newton’s method to $f(x) = \frac{1}{x} - b$, which gives

$$x_{k+1} = x_k(2 - bx_k),$$

and each step is 2 multiplications.

To see how fast this converges, let $e_k = 1 - bx_k$ be the error. Then

$$1 - bx_{k+1} = 1 - bx_k(2 - bx_k) = 1 - 2bx_k + (bx_k)^2 = (1 - bx_k)^2,$$

so $e_{k+1} = e_k^2$. If $e_k < 2^{-m}$ then $e_{k+1} < 2^{-2m}$, so the number of correct bits doubles at every step and we need only $O(\log n)$ steps.

The cost does not blow up, because at step k we only have 2^k correct bits, so we only need to compute to 2^k bits of precision. The working precisions are $1, 2, 4, \dots, n$, so the total is

$$M(1) + M(2) + M(4) + \dots + M(n) \leq 2M(n)$$

by the geometric sum, since superlinearity makes the last term dominate.

Finally $\lfloor a/b \rfloor$ is $a \times (1/b)$, one more multiplication, plus a correction of at most 1. Hence $D(n) = O(M(n))$.

Second direction: $M(n) = O(D(n))$

Multiplication reduces to squaring, since

$$a \cdot b = \frac{(a+b)^2 - a^2 - b^2}{2},$$

which is three squarings, two subtractions and one shift. So $M(n) = O(S(n) + n)$, where S is the cost of squaring.

Squaring in turn reduces to one division. Take the geometric series

$$\frac{1}{2^k - a} = \frac{2^{-k}}{1 - a2^{-k}} = \sum_{j \geq 0} a^j 2^{-k(j+1)}$$

and multiply by 2^{4k} :

$$\frac{2^{4k}}{2^k - a} = 2^{3k} + a 2^{2k} + a^2 2^k + a^3 + \frac{a^4 2^{-k}}{1 - a 2^{-k}}.$$

Choose $k = 4n$. Since $a < 2^n$ we get $a^4 < 2^{4n} = 2^k$, so the last term is less than 2. And each of a, a^2, a^3 is smaller than 2^k , so each sits inside its own k -bit block without overlapping the others.

So one division produces a number whose binary expansion literally contains a, a^2 and a^3 in consecutive blocks. We read off the bits in $[k, 2k)$ and that is a^2 . The last term can only disturb the lowest bit, which lies in the a^3 block, so a^2 is untouched. Reading off a block is a shift and a mask, which is $O(n)$. Hence $S(n) = O(D(n) + n)$, and therefore $M(n) = O(D(n))$.

Conclusion

$$D(n) = O(M(n)) \quad \text{and} \quad M(n) = O(D(n)) \quad \implies \quad \boxed{D(n) = \Theta(M(n))}$$

So the two are equivalent up to constant factors.

Diagram and L^AT_EX code generation credit: Claude
